

Elementary Operations on Quasi-Toeplitz Matrices

PCCA6

Ali Arda BARUT
Rafael LLAMAS

May 22, 2026

Contents

1	Introduction	2
1.1	Foundations of Toeplitz and quasi-Toeplitz Structures	2
1.1.1	Toeplitz Matrices	2
1.1.2	Displacement Operators	2
1.1.3	Generator Matrices and Reconstruction	3
1.1.4	Displacement Operators	4
2	Operation	4
2.1	Addition	4
2.1.1	Introduction	4
2.1.2	Exemple	4
2.1.3	Benchmark	5
2.2	Multiplication	5
2.2.1	Introduction	5
2.2.2	Multiplication Vecteur	6
2.2.3	Exemple	7
2.2.4	Benchmark	8
3	Generator Compression	8
4	Inversion	10
4.1	Algorithm	10
4.2	Split and Pack quadrants from displacement generators	11
4.2.1	Splitting	11
4.2.2	Packing	12
4.3	Benchmarks	13

1 Introduction

In computational numerical linear algebra, dense matrices often present a computational bottleneck, scaling quadratically in terms of memory usage ($O(nm)$) for a matrix with n rows and m columns and cubically in terms of time complexity ($O(n^3)$) for standard operations. However, in many real-world life applications such as signal processing[3][7] and classical simulation of quantum systems[9], these dense matrices are not totally random. The elements are arranged in repeating, deterministic patterns. Designing algorithms that take advantage of this pattern can bypass the fundamental limitations of dense linear algebra.

To exploit these patterns, we apply a displacement operation to determine how much a matrix deviates from a perfect diagonal structure. For a highly structured matrix, this displacement results in a low rank. This demonstrates that it is possible to represent a dense matrix using two smaller, more compact matrices. Performing operations on these matrices reduces both storage and computation requirements. The aim of this project is to explore and implement elementary operations on quasi-Toeplitz matrices using these displacement operators.

1.1 Foundations of Toeplitz and quasi-Toeplitz Structures

In order to mathematically formalise the concepts introduced above, we must first define these structured patterns. This section outlines the properties of Toeplitz and quasi-Toeplitz matrices and introduces the displacement operators necessary for extracting their generators.

1.1.1 Toeplitz Matrices

A Toeplitz matrix is characterized by the property that all elements along any given diagonal are identical. For a matrix $A \in \mathbb{K}^{n \times m}$, it satisfies the condition $A_{i,j} = A_{i+1,j+1}$. In other words, a Toeplitz matrix is one in which each element in the first row and first column descends diagonally. It can be represented by a sequence of elements $\{a_k\}$ such that:

$$A = \begin{pmatrix} a_0 & a_{-1} & \cdots & & & & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & & & & a_{1-(m-1)} \\ \vdots & a_1 & a_0 & \ddots & & & a_{2-(m-1)} \\ \vdots & & \ddots & \ddots & \ddots & & \vdots \\ a_{n-1} & \cdots & \cdots & a_1 & a_0 & a_{-1} & \cdots & a_{(n-1)-(m-1)} \end{pmatrix}$$

$$A_{i,j} = a_{i-j}, \quad 0 \leq i < n, \quad 0 \leq j < m$$

1.1.2 Displacement Operators

Despite the perfect Toeplitz matrix defined above, the pattern of real-world operations is often slightly corrupted. Matrices that deviate slightly from pure Toeplitz form are referred to as quasi-Toeplitz or Toeplitz-like matrices. Operational evaluation of these matrices is achieved through displacement. For a matrix $A \in \mathbb{K}^{n \times m}$, we define the displacement matrix $\phi_+(A)$ using:

$$\phi_+(A) = A - ZAZ^T \tag{1}$$

where Z represents the lower shift matrix and Z^T represents the upper shift matrix. Essentially, Z is a matrix with ones on the first sub-diagonal and zeros everywhere else:

$$Z = \begin{pmatrix} 0 & & & & \\ 1 & & & & \\ & \ddots & & & \\ & & 1 & & 0 \end{pmatrix} \in \mathbb{K}^{n \times n} \quad Z^T = \begin{pmatrix} 0 & 1 & & & \\ & & \ddots & & \\ & & & 1 & \\ & & & & 0 \end{pmatrix} \in \mathbb{K}^{m \times m}$$

For a generic Toeplitz matrix of size $n \times m$, this subtraction results in zeros everywhere except for the first row and the first column. This will allow us to represent the matrix using only $n + m$ elements.

$$\phi_+(A) = \begin{pmatrix} a_0 & a_{-1} & a_{-2} & \cdots & a_{-(m-1)} \\ a_1 & 0 & 0 & \cdots & 0 \\ a_2 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & 0 & 0 & \cdots & 0 \end{pmatrix} \in \mathbb{K}^{n \times m}$$

While the definition in equation (1) suggests a matrix on matrix subtraction with overhead, the structured nature of A allows for a more efficient computation. By leveraging the Toeplitz structure, displacement $\phi_+(A)$ can be computed via a loop:

$$(\phi_+(A))_{i,j} = \begin{cases} A_{i,j} & \text{if } i = 0 \text{ or } j = 0 \\ A_{i,j} - A_{i-1,j-1} & \text{otherwise} \end{cases} \quad (2)$$

In practice, we enable optimization through pointer arithmetics. By iterating from the bottom-right index (n, m) toward the origin $(0, 0)$, the displacement can be calculated in-place, resulting a complexity bound of $O(nm)$.

1.1.3 Generator Matrices and Reconstruction

The displacement matrix $\phi_+(A)$ physically occupies an $n \times m$ dense layout with the 0s, its low displacement rank ($\alpha \leq 2$ for Toeplitz) allows A to be compressed and stored via a pair of smaller matrices known as generators, denoted as $G \in \mathbb{K}^{n \times \alpha}$ and $H \in \mathbb{K}^{m \times \alpha}$, which satisfy:

$$\phi_+(A) = GH^T \quad (3)$$

To reconstruct the uncompressed operator A directly from its generator columns g_k and h_k (for $k = 1, \dots, \alpha$), we formalize the families of lower and upper triangular Toeplitz operators.

For a column vector $v = [v_0, v_1, \dots, v_{n-1}]^T \in \mathbb{K}^n$, we define the lower triangular Toeplitz operator $L_n(v) \in \mathbb{K}^{n \times n}$ as the square matrix. Let $U_{n,m}(w) \in \mathbb{K}^{n \times m}$ be the rectangular upper triangular Toeplitz operator generated by a vector $w = [w_0, w_1, \dots, w_{m-1}]^T \in \mathbb{K}^m$. This operator is formed by taking the first n rows of the fully square $m \times m$ upper triangular matrix $L_m(w)^T$:

$$L_n(v) = \begin{pmatrix} v_0 & 0 & 0 & \cdots & 0 \\ v_1 & v_0 & 0 & \cdots & 0 \\ v_2 & v_1 & v_0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \cdots & v_1 & v_0 \end{pmatrix} \quad U_{n,m}(w) = \begin{pmatrix} w_0 & w_1 & w_2 & \cdots & \cdots & w_{m-1} \\ 0 & w_0 & w_1 & \cdots & \cdots & w_{m-2} \\ 0 & 0 & w_0 & \cdots & \cdots & w_{m-3} \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & w_0 & \cdots \end{pmatrix}$$

By denoting the individual columns of the generators as $G = [g_1, \dots, g_\alpha]$ and $H = [h_1, \dots, h_\alpha]$, the displacement inversion theorem [1] establishes that the full matrix A can be synthesized as a sum of products of these triangular structures:

$$GH^T = \phi_+(A) \iff A = \sum_{k=1}^{\alpha} L_n(g_k) U_{n,m}(h_k) \quad (4)$$

1.1.4 Displacement Operators

We define two displacements operators:

$$\begin{aligned} \phi_+(A) &= A - ZAZ^T \\ \phi_-(A) &= A - Z^T AZ \end{aligned}$$

where Z is the downshift matrix defined in the previous section. The two operators differ in the direction of the operation: ϕ_+ operates by Z (a downshift), while ϕ_- operates by Z^T (an upshift). For a Toeplitz matrix A , both operators gives a low-rank result, with $\text{rank}(\phi_{\pm}(A)) \leq 2$. Computationally, it follows the same path as the calculation of $\phi_+(A)$. Main difference being we can run this time in increasing order:

$$\phi_-(A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = n - 1 \text{ or } j = m - 1 \\ A_{i,j} - A_{i+1,j+1} & \text{otherwise} \end{cases} \quad (5)$$

Where n and m are respectfully the number of rows and columns of matrix A .

2 Operation

2.1 Addition

2.1.1 Introduction

With the displacement generators, we can simplify the addition of 2 matrices and thus reduce its cost by concatenating the generators of the 2 matrices. From the property 10.7 and 10.8, we can pass from $O(n^2)$ for the naive algorithm to $O(\alpha \times M(n))$ for the version that we implement. We need $A=(T,U)$ and $B=(G,H)$

Algorithm 1 Generator Addition

```

1: function GR_MAT_MUL_GENERATOR( $T, U, G, H$ )
2:    $G_C \leftarrow \text{GR\_MAT\_CONCAT\_HORIZONTAL}(T, G)$ 
3:    $H_C \leftarrow \text{GR\_MAT\_CONCAT\_HORIZONTAL}(U, H)$ 
4:   return  $G_C, H_C$ 
5: end function

```

2.1.2 Example

We takes A and B , two randoms matrices, C the result we need to get of A and B . Every matrices are in $Z/_{13}Z$.

$$A = \begin{pmatrix} 0 & 7 & 10 \\ 5 & 0 & 7 \\ 0 & 5 & 0 \end{pmatrix}, B = \begin{pmatrix} 9 & 12 & 4 \\ 0 & 9 & 12 \\ 3 & 0 & 9 \end{pmatrix}, C = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \quad (6)$$

If we takes displacement generator of $A = (T,U)$ and $B = (G,H)$

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, U = \begin{pmatrix} 5 & 0 \\ 0 & 7 \\ 0 & 10 \end{pmatrix}, G = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 9 & 1 \end{pmatrix}, H = \begin{pmatrix} 9 & 0 \\ 12 & 9 \\ 4 & 3 \end{pmatrix} \quad (7)$$

and we do the concatenation of T,G and U,H to obtain RES_G and RES_H (we have to do the concatenation in the order that I cited the elements).

$$RES_G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 9 & 1 \end{pmatrix}, RES_H = \begin{pmatrix} 5 & 0 & 9 & 0 \\ 0 & 7 & 12 & 9 \\ 0 & 10 & 4 & 3 \end{pmatrix} \quad (8)$$

The matrix that we get D :

$$D = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \quad (9)$$

We can see that the matrix D is equal to C so our way to add the 2 matrices is correct in this case.

2.1.3 Benchmark

The benchmark obtained highlights the following results:

n	Generators (ms)	Flint Dense (ms)
128	3.577e-03	1.373e-02
1024	3.205e-02	2.52
4096	1.148e-01	29.18

Table 1: Median runtimes over 10 runs

(ici courbe des temps) If we compare matrices of 128,1024 and 4096,we got : $T(128) \div 128 \approx 2.79e - 05$, $T(1024) \div 1024 \approx 3.13e - 05$, $T(4096) \div 4096 \approx 2.80e - 05$. We can see that the time of addition is linear in the size of the matrix, so we can conclude that the complexity of the addition is $O(\alpha n)$.

2.2 Multiplication

2.2.1 Introduction

With displacement generators we also can simplify the multiplication of 2 matrices and thus reduce its cost by concatenating the generators of the 2 matrices. - The displacement generators $A=(T,U)$ and $B=(G,H)$

$$- W = Z \times A \times Z^t \times G$$

$$- V = B^t \times U$$

- a (resp. b) a vector which is the last column of $Z \times A$ (resp. $Z \times B^t$)

This allows us to pass from $O(n^3)$ for the naive algorithm to $O(\alpha^2 \times M(n))$ for the version we implement.

Algorithm 2 Generator Multiplication

```

1: function GR_MAT_MUL_GENERATOR( $T, U, G, H$ )
2:    $W_{Temp} \leftarrow \text{GR\_MAT\_APPLY\_ZT}(G)$ 
3:    $W \leftarrow \text{GR\_MAT\_MUL\_VECTOR}(T, U, W_{Temp})$ 
4:    $W \leftarrow \text{GR\_MAT\_APPLY\_Z}(W)$ 

5:    $V \leftarrow \text{GR\_MAT\_MUL\_VECTOR}(H, G, U)$ 

6:    $a \leftarrow \text{GR\_MAT\_MUL\_VECTOR}(T, U, \text{LastCol})$ 
7:    $a \leftarrow \text{GR\_MAT\_APPLY\_Z}(a)$ 

8:    $b \leftarrow \text{GR\_MAT\_MUL\_VECTOR}(H, G, \text{LastCol})$ 
9:    $b \leftarrow \text{GR\_MAT\_APPLY\_Z}(b)$ 
10:   $b \leftarrow \text{GR\_MAT\_NEG}(b)$ 

11:   $G_C \leftarrow \text{GR\_MAT\_CONCAT\_HORIZONTAL}(T, W, a)$ 
12:   $H_C \leftarrow \text{GR\_MAT\_CONCAT\_HORIZONTAL}(V, H, b)$ 
13:  return  $G_C, H_C$ 
14: end function

```

We use the vector

$$\text{LastCol} = \begin{pmatrix} 0 \\ 0 \\ \cdot \\ \cdot \\ 0 \\ 1 \end{pmatrix} \quad (10)$$

in order to retrieve the last column of the matrix, this vector is composed of 0 except for the last element of the column which is fixed to 1.

2.2.2 Multiplication Vecteur

Perform the matrix multiplication with their displacement generators needs a function that multiplies a vector with displacement generators to avoid recomposing the matrices.

$$C = \sum_{k=1}^{\alpha} L(g_k) U(h_k) V \quad (11)$$

To do that, we takes the formula for the reconstruction and we transform our displacement generators into polynomials and make every operation into them.

Algorithm 3 Multiplication de Matrice Structurée par un Vecteur/Matrice

```

1: function GR_MAT_MUL_VECTOR( $G, H, V$ )
2:    $n \leftarrow nbRows(G)$ 
3:    $m \leftarrow nbRows(H)$ 
4:   for  $j \leftarrow 0$  to  $nbCols(V)$  do
5:      $P_v \leftarrow \sum_{r=0}^{m-1} V[r, j]x^r$ 
6:     for  $k \leftarrow 0$  to  $nbCols(G)$  do
7:        $P_g \leftarrow \sum_{r=0}^{n-1} G[r, k]x^r$ 
8:        $P_h \leftarrow \sum_{r=0}^{m-1} H[r, k]x^r$ 
9:        $P_{h\_rev} \leftarrow \text{Reverse}(P_h, m)$ 
10:       $P_{tmp} \leftarrow P_{h\_rev}P_v$ 
11:       $P_{tmp\_shift} \leftarrow P_{tmp} \gg (m-1)$ 
12:       $P_{res} \leftarrow P_g P_{tmp\_shift}$ 
13:      for  $i \leftarrow 0$  to  $\text{Length}(P_{tmp2}) - 1$  do
14:         $C[i, j] \leftarrow C[i, j] + \text{Coefficient}(P_{res}, i)$ 
15:      end for
16:    end for
17:  end for
18:  return  $Res$ 
19: end function

```

2.2.3 Exemple

We takes A and B, two randoms matrices, C the result we need to get of A and B. Every matrices are in $Z/_{13}Z$.

$$A = \begin{pmatrix} 10 & 5 & 8 \\ 5 & 10 & 5 \\ 3 & 5 & 10 \end{pmatrix}, B = \begin{pmatrix} 0 & 3 & 0 \\ 3 & 0 & 3 \\ 0 & 3 & 0 \end{pmatrix}, C = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \quad (12)$$

If we takes displacement generator of A = (T,U) and B = (G,H)

$$T = \begin{pmatrix} 1 & 0 \\ 7 & 1 \\ 12 & 11 \end{pmatrix}, U = \begin{pmatrix} 10 & 0 \\ 5 & 4 \\ 8 & 9 \end{pmatrix}, G = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, H = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix} \quad (13)$$

We need to calculate W and V before concatenating the generators of A and B to obtain the displacement generator of D.

$$W = \begin{pmatrix} 0 & 0 \\ 10 & 0 \\ 5 & 0 \end{pmatrix}, V = \begin{pmatrix} 2 & 12 \\ 2 & 1 \\ 2 & 12 \end{pmatrix} \quad (14)$$

Then we do the concatenation of T,W,a and V,H,-b to obtain RES_G and RES_H (we have to do the concatenation in the order that I cited the elements)

$$RES_G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 7 & 1 & 10 & 0 & 8 \\ 12 & 11 & 5 & 0 & 5 \end{pmatrix}, RES_H = \begin{pmatrix} 2 & 12 & 3 & 0 & 0 \\ 2 & 1 & 0 & 3 & 0 \\ 2 & 12 & 0 & 0 & 10 \end{pmatrix} \quad (15)$$

La matrice que l'on a obtenue D :

$$D = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \quad (16)$$

We can see that the matrix D is equal to C so our way to multiply the 2 matrices is correct in this case.

2.2.4 Benchmark

The benchmark obtained highlights the following results:

n	Generators (ms)	Flint Dense (ms)	Vector (ms)	Vector Flint (ms)
128	6.289e-01	3.032	1.221e-01	3.217e-02
1024	3.736	984.2	6.383e-01	1.554
4096	12.42	39810	2.063	17.63

Table 2: Median runtimes over 10 runs

(ici courbe des temps) For the multiplication with vector, if we compare matrices of 128, 1024 and 4096, we got : $T(128) \div 128 \approx 9.5e-04$, $T(1024) \div 1024 \approx 6.2e-04$, $T(4096) \div 4096 \approx 5e-04$. We can see that the time of multiplication with vector is linear in the size of the matrix, so we can conclude that the complexity of the multiplication with vector is $O(\alpha n)$.

For the multiplication with generators, if we compare matrices of 128, 1024 and 4096, we got : $T(128) \div 128^2 \approx 4.91e-03$, $T(1024) \div 1024^2 \approx 3.65e-03$, $T(4096) \div 4096^2 \approx 3.03e-03$. We can see that the time of multiplication with generators is linear in the square of the size of the matrix, so we can conclude that the complexity of the multiplication with generators is $O(\alpha^2 M(n))$.

3 Generator Compression

Matrices $G \in \mathbb{R}^{n \times \alpha}$ and $H \in \mathbb{R}^{n \times \alpha}$ representing a displacement matrix $\nabla A = GH^T$, where the actual rank of ∇A is $r < \alpha$.

Goal: Find generators $G' \in \mathbb{R}^{n \times r}$ and $H' \in \mathbb{R}^{n \times r}$ s.t. the product is the same:

$$G'H'^T = GH^T$$

We will achieve this by independently compression both matrices and applying the inverse of the operations done to the other.

Algorithm 4 Generator Compression

```

1: function GR_MAT_GENERATOR_COMPRESS( $G_A, H_A$ )
2:    $G_1, H_1 \leftarrow$  GR_MAT_GENERATOR_COMPRESS_AUX( $G_A, H_A$ )
3:    $H_D, G_D \leftarrow$  GR_MAT_GENERATOR_COMPRESS_AUX( $H_1, G_1$ )
4:   return  $G_D, H_D$ 
5: end function

```

Algorithm 5 Generator Compression Auxiliary

```

1: function GR_MAT_GENERATOR_COMPRESS_AUX( $G, H$ )
2:    $P, L, U \leftarrow \text{LU\_Decomposition}(G^T)$   $\triangleright PG^T = LU$ 
3:    $r \leftarrow \text{Number of non-zero rows in } U$   $\triangleright \text{rank}$ 
4:   if  $r < 1$  then
5:     return Generators of a 0 matrix
6:   end if
7:    $G_D \leftarrow (U_{[0:r-1],*})^T$ 
8:    $H_D \leftarrow HP^T \times L_{*,[0:r-1]}$ 
9:   return  $G_D, H_D$ 
10: end function

```

Note that calling `GR_MAT_GENERATOR_COMPRESS_AUX` twice with swapped arguments (and variables) is valid because each call preserves the product GH^T , as can be verified from the LU factorization.

Proof. We show that the second call to the auxiliary function preserves the generator product, meaning $H_D G_D^T = H_1 G_1^T$.

First argument is H_1 , the LU decomposition is performed on H_1^T giving $PH_1^T = LU$. Since U has rank r , the product only depends on the first r columns and rows: $LU = L_{*,[0:r-1]} U_{[0:r-1],*}$. Starting from $PH_1^T = LU$ and multiplying both sides on the left by P^T :

$$\begin{aligned} H_1^T &= P^T L_{*,[0:r-1]} U_{[0:r-1],*} \\ H_1 &= (U_{[0:r-1],*})^T L_{*,[0:r-1]}^T P \end{aligned}$$

Based on the swapped inputs to the auxiliary function, returned generators are given as $H_D = (U_{[0:r-1],*})^T$ and $G_D = G_1 P^T L_{*,[0:r-1]}$. Taking the transpose of G_D :

$$G_D^T = (G_1 P^T L_{*,[0:r-1]})^T = L_{*,[0:r-1]}^T P G_1^T$$

For $H_D G_D^T$, we see H_1 form:

$$\begin{aligned} H_D G_D^T &= (U_{[0:r-1],*})^T \left(L_{*,[0:r-1]}^T P G_1^T \right) \\ &= \left((U_{[0:r-1],*})^T L_{*,[0:r-1]}^T P \right) G_1^T \\ &= H_1 G_1^T. \end{aligned}$$

Since $H_D G_D^T = H_1 G_1^T$, transposing gives us $G_D H_D^T = G_1 H_1^T$. $\square$

Computing the LU factorization of the transposed generator $G^T \in \mathbb{K}^{\alpha \times n}$ requires $\mathcal{O}(\alpha^2 n)$ operations. Extracting U and applying the permutation matrix P^T to H is in $\mathcal{O}(\alpha n)^1$. Computing $H_D = HP^T \times L_{*,[0:r-1]}$ contains a multiplication of a $n \times \alpha$ matrix by a $\alpha \times r$ matrix. Using standard matrix multiplication, this takes $\mathcal{O}(n\alpha r)$ operations. Our implementation uses loops to extract the selected amount of rows and columns, but their complexities are strictly bounded by $(n\alpha r)$.

We can generalize this complexity: for any generator matrices in $\mathbb{K}^{n \times m}$ compressing to rank r , the overall time complexity for the generator compression is dominated by the matrix multiplication and is bounded by $\mathcal{O}(rnm)$.

¹In FLINT[8], the permutation matrix is represented as an array of indices. We do a swapping operation rather than a full matrix multiplication.

4 Inversion

4.1 Algorithm

We present an implementation of Strassen's inversion method specifically adapted for displacement generators. Strassen's approach relies on recursive block-matrix partitioning and Schur complements,

Algorithm 7 Strassen-type Inversion for Displacement Generators (adapted from [1])

```

1: procedure STRASSENINVERSEGENERATORS( $G_A, H_A$ )
2:   if  $n = 1$  then                                     ▷ Step 1: Base Case
3:     return  $G_D H_D$  representing the inverse of the scalar value of  $A$  from  $G_A, H_A$ 
4:   end if

5:    $G_a, H_a, \dots, G_d, H_d \leftarrow \text{SPLITQUADRANTS}(G_A, H_A)$   ▷ Step 2: Split into Quadrants
6:    $G_e, H_e \leftarrow \text{STRASSENINVERSEGENERATORS}(G_a, H_a)$       ▷ Step 3: Recursion#1
                                                                ▷  $e := a^{-1}$ 

7:    $G_{ce}, H_{ce} \leftarrow \text{COMPRESS}(\text{MUL}(G_c, H_c, G_e, H_e))$   ▷ Step 4: Schur Complement
8:    $G_{eb}, H_{eb} \leftarrow \text{COMPRESS}(\text{MUL}(G_e, H_e, G_b, H_b))$ 
9:    $G_{ceb}, H_{ceb} \leftarrow \text{COMPRESS}(\text{MUL}(G_c, H_c, G_{eb}, H_{eb}) \times -1)$ 
10:   $G_S, H_S \leftarrow \text{COMPRESS}(\text{ADD}(G_d, H_d, G_{ceb}, H_{ceb}))$     ▷  $S := d - ceb$ 
                                                                ▷ Step 5: Recursion#2
                                                                ▷  $t := S^{-1}$ 

11:   $G_t, H_t \leftarrow \text{STRASSENINVERSEGENERATORS}(G_S, H_S)$ 

12:   $G_{ebt}, H_{ebt} \leftarrow \text{COMPRESS}(\text{MUL}(G_{eb}, H_{eb}, G_t, H_t))$   ▷ Step 6: Strassen Assembly
13:   $G_{tce}, H_{tce} \leftarrow \text{COMPRESS}(\text{MUL}(G_t, H_t, G_{ce}, H_{ce}))$ 
14:   $G_{ebtce}, H_{ebtce} \leftarrow \text{COMPRESS}(\text{MUL}(G_{ebt}, H_{ebt}, G_{ce}, H_{ce}))$ 

15:   $G_x, H_x \leftarrow \text{COMPRESS}(\text{ADD}(G_e, H_e, G_{ebtce}, H_{ebtce}))$     ▷  $x := e + ebtce$ 
16:   $G_y, H_y \leftarrow \text{NEGATE}(G_{ebt}, H_{ebt})$                         ▷  $y := -ebt$ 
17:   $G_z, H_z \leftarrow \text{NEGATE}(G_{tce}, H_{tce})$                         ▷  $z := -tce$ 

18:   $G_{Inv}, H_{Inv} \leftarrow \text{PACKQUADRANTS}(G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t)$   ▷ Step 7: Pack
19:  return  $\text{COMPRESS}(G_{Inv}, H_{Inv})$ 
20: end procedure

```

This algorithm is based on Algorithm 10.1 from [1, Algorithm 10.1]. Notice that we only operate on Φ_+ displacement and bypassing Φ_- generators.

Proof. We proceed by induction on the size of the matrix $n = 2k$. Let $\Phi_+(X) = X - ZXZ^T$ be the displacement operator.

Base Case ($n = 1$): For a scalar matrix $A = [a]$, the operator Z is a 1×1 zero matrix. The displacement of A is $\Phi_+(A) = A - 0 \cdot A \cdot 0 = A$.

Inductive Step: Consider a matrix A of size $2k \times 2k$ represented by Φ_+ generators. Inversion relies on splitting the sub-blocks a, b, c, d which are represented by Φ_+ displacements, recursive calls to invert top-left block a ($e = a^{-1}$) and another for the Schur complement $t = S^{-1}$ and each outputs a Φ_+ generator. Multiplication and addition algorithms are taking and returning

a generator of Φ_+ displacement. Each computation and the final assembly remain in Φ_+ displacement.

Because the base case returns the inverse to Φ_+ generators, and the arithmetics are on Φ_+ generator operations, by induction, we maintain Φ_+ displacement during the entire execution. $\square$

4.2 Split and Pack quadrants from displacement generators

Any divide and conquer algorithm will divide the problem to create sub-problems to economise in complexity. This algorithm calls for a dense matrix A be split into four quadrants:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Since we don't have direct access to the dense matrix A , any splitting and packing operation done will be on the generators G_A and H_A which represent the displacement matrix. When we attempt to evaluate the local displacement of a sub-block (for example, $\Phi_+(d)$), we cannot simply extract the rows from the global generators G_A and H_A . The global generators are corrupted by the boundary elements of a that shifted into b and c . We must correct these boundary crossings which we call bleeding.

4.2.1 Splitting

During the split phase, we extract the local generators for sub-blocks a, b, c, d . We begin by splitting the global generators $G_A, H_A \in \mathbb{R}^{n \times r}$ into top and bottom blocks of sizes $n_1 = \text{ceil}(n/2)$ and $n_2 = \text{floor}(n/2)$:

$$G = \begin{pmatrix} G_{\text{TOP}} \\ G_{\text{BOTTOM}} \end{pmatrix} \quad H = \begin{pmatrix} H_{\text{TOP}} \\ H_{\text{BOTTOM}} \end{pmatrix}$$

Block a: This block receives no shifted data from other quadrants. Its generators are the global top halves:

$$G_a = G_{\text{TOP}}, \quad H_a = H_{\text{BOTTOM}}$$

Block b: The last column of block a , denoted v_a , shifts right into the first column of block b . To correct this bleed, we must reconstruct v_a from the global generators. Naively evaluating the displacement sum for a boundary vector requires $O(n^2 r)$ operations. However, this evaluation is equivalent to computing the coefficients of a polynomial multiplication. By packing the columns of G and the reversed columns of H into polynomials, we extract v_a with polynomial multiplication in $O(r \cdot \mathcal{M}(n))$ time. Once extracted, we shift v_a down using the operator Z_{n_1} and append it to G_{TOP} . To apply this correction only to the first column of b , we append the vector $e_0^{(n_2)} = (1, 0, \dots, 0)^T$ to H_{bot} :

$$G_b = (G_{\text{TOP}} \quad Z_{n_1} v_a), \quad H_b = (H_{\text{BOTTOM}} \quad e_0^{(n_2)})$$

Block c: Bleeding occurs to c from above via the last row of a , denoted r_a . Using the same polynomial operation, we recover r_a . Because H generators represent the transposed displacement components, we apply the shift Z_{n_1} to r_a and append it to H_{TOP} :

$$G_c = (G_{\text{BOTTOM}} \quad e_0^{(n_2)}), \quad H_c = (H_{\text{TOP}} \quad Z_{n_1} r_a)$$

Block d: Block d is exposed to bleeding from all directions: the last column of c (v_c) shifting right, the last row of b (r_b) shifting down, and the exact bottom-right scalar of a (s_a) shifting

diagonally down-right. We extract v_c and r_b using polynomial operation. Because s_a is a single cell, polynomial overhead is unnecessary; it is computed directly in (nr) time. Scaling the vector $e_0^{(n_2)}$ by s_a and addint to the shifted vectors:

$$G_d = \begin{pmatrix} G_{\text{BOTTOM}} & s_a e_0^{(n_2)} + Z_{n_2} v_c & e_0^{(n_2)} \end{pmatrix}, \quad H_d = \begin{pmatrix} H_{\text{BOTTOM}} & e_0^{(n_2)} & Z_{n_2} r_b \end{pmatrix}$$

4.2.2 Packing

The pack operation reverts the changes of the splitting phase. After computing the inverse sub-blocks (denoted as x, y, z, t), our goal is to assemble the global generators G_{INV} and H_{INV} for the full inverse matrix A^{-1} . If we were to concatenate directly the local generators, the result would assume that the boundaries between the sub-blocks are filled with zeros. Because the global operator Φ_+ shifts elements, we must correct the bleeding to restore data integrity. Just as we calculated boundaries during the split, we must extract the boundary vectors of the local inverse blocks. Since we are now operating on isolated sub-block generators, the coordinates for each reconstruction are local and starts from zero.

To apply these corrections, we construct a large set of global generators. We horizontally concatenate the local generators of x, y, z, t into their respective quadrants and append four specific boundary correction columns (c_1, c_2, c_3, c_4). We define these global assembled generators as block matrices:

$$G_{RES} = \begin{pmatrix} G_x & G_y & 0 & 0 & -Z_{n_1} v_x & 0 & 0 & 0 \\ 0 & 0 & G_z & G_t & 0 & -e_0^{(n_2)} & -(s_x e_0^{(n_2)} + Z_{n_2} v_z) & -e_0^{(n_2)} \end{pmatrix}$$

$$H_{RES} = \begin{pmatrix} H_x & 0 & H_z & 0 & 0 & Z_{n_1} r_x & 0 & 0 \\ 0 & H_y & 0 & H_t & e_0^{(n_2)} & 0 & e_0^{(n_2)} & Z_{n_2} r_y \end{pmatrix}$$

Notice that the boundary correction columns in G_{RES} carry negative signs. The reconstruction is going to add the values diagonally, to correct bleeding, we must subtract the the corrections. To prove the correctness of these columns, we compute the global displacement reconstruction by multiplying $\Phi_+(A^{-1}) = G_{RES} H_{RES}^T$:

$$G_{RES} H_{RES}^T = \begin{pmatrix} G_x H_x^T & G_y H_y^T - Z_{n_1} v_x (e_0^{(n_2)})^T \\ G_z H_z^T - e_0^{(n_2)} (Z_{n_1} r_x)^T & G_t H_t^T - Z_{n_2} v_z (e_0^{(n_2)})^T - e_0^{(n_2)} (Z_{n_2} r_y)^T - s_x e_0^{(n_2)} (e_0^{(n_2)})^T \end{pmatrix}$$

The isolated terms ($G_x H_x^T, G_y H_y^T$, etc.) represent the isolated local displacements of the sub-matrices. The subtracted terms are the bleeds. For example, the term $-Z_{n_1} v_x (e_0^{(n_2)})^T$ subtracts the shifted right column of block x as it crosses into the first column of block y .

This structural reassembly exposes a vulnerability. Let $\alpha_x, \alpha_y, \alpha_z, \alpha_t$ be the displacement ranks of the sub-quadrants. The rank of our assembled global generators is:

$$\text{Rank}_{RES} = \alpha_x + \alpha_y + \alpha_z + \alpha_t + 4$$

Passing this uncompressed matrix up to the next recursion level would cause the displacement rank to explode, ruining the optimisations we aimed for at the first place. We call the compression seen in section 3 to keep the algorithm efficient.

n	Generators (ms)	Flint Dense (ms)
512	30.14	79.19
1024	70.42	545.50
2048	164.10	3839.00
4096	374.00	26610.00

Table 3: Median runtimes of inversion over 5 runs

α	$n = 512$ (ms)	$n = 1024$ (ms)	$n = 2048$ (ms)
2	33.22	76.98	180.00
3	46.15	109.50	253.10
4	60.58	147.30	340.20
5	77.06	188.30	440.00
10	182.30	455.90	1098.00

Table 4: Median runtimes of generator inversion scaling by rank over 5 runs

4.3 Benchmarks

The benchmarks show great improvement on generator-based Strassen inversion. Table 3 demonstrates the advantage over FLINT’s dense inversion of $O(n^3)$ bound. While split and pack phases operates in $O(\alpha \cdot \mathcal{M}(n))$, the full recursive inversion involves multiplying structured sub-generators which makes this algorithm bounded by $O(\alpha^2 \mathcal{M}(n) \log n)$,

While this implementation is great for large matrices with a bounded rank, the overhead makes the algorithm computationally disadvantageous for smaller matrices compared to standard dense inversion. We have observed the threshold generally between matrices of sizes 256 and 512.

As detailed by Victor Y. Pan in *Structured Matrices and Polynomials* [6, Chapter 5], an alternative is the Morf-Bitmead-Anderson (MBA) algorithm. Originally proposed by Morf (1980) and Bitmead and Anderson (1980), it achieves $O(n \log^2 n)$ complexity for Toeplitz-like matrices. Because it is also a divide-and-conquer method, this approach still requires calculations of sub-quadrants and continuous generator compression. To bypass the need to split the matrix, one should prefer to implement the Newton iterative version. Pan dedicates analysis to this approach in [6, Chapters 6 and 7], which is based on the Schulz iteration: $X_{k+1} = X_k(2I - AX_k)$.

If the objective is to solve a linear system $Ax = b$, computing the full inverse A^{-1} introduces unnecessary computations. Instead, one can take direct advantage of the structured properties of A to solve the system more efficiently. Methods such as the Levinson-Durbin algorithm [4], avoid inversion and solve Toeplitz systems in $O(n^2)$ operations. Iterative Krylov subspace methods, the Preconditioned Conjugate Gradient (PCG) method [2] use FFT to compute matrix-vector products, achieving $O(n \log n)$ complexity per iteration. These iterative methods they rely on numerical approximation and convergence tolerances. For an exact deterministic algebraic solution, there is an alternative by resolving via M-Padé approximations[5], which achieves a deterministic $\tilde{O}(\alpha^{\omega-1}n)$ complexity.

References

- [1] Alin Bostan, Frédéric Chyzak, Marc Giusti, Romain Lebreton, Grégoire Lecerf, Bruno Salvy, and Éric Schost. Algorithmes efficaces en calcul formel, August 2017. 686 pages. Édition 1.0. URL: <https://hal.archives-ouvertes.fr/AECF/>.
- [2] Tony F. Chan. An optimal circulant preconditioner for toeplitz systems. *SIAM Journal on Scientific and Statistical Computing*, 9(4):766–771, 1988. arXiv:<https://doi.org/10.1137/0909051>, doi:10.1137/0909051.
- [3] Ignat Domanov and Lieven De Lathauwer. Generic uniqueness of a structured matrix factorization and applications in blind source separation. *IEEE Journal of Selected Topics in Signal Processing*, 10(4):701–711, 2016. doi:10.1109/JSTSP.2016.2526971.
- [4] James Durbin. The fitting of time-series models. *Revue de l’Institut International de Statistique*, pages 233–244, 1960.
- [5] Sara Khichane and Vincent Neiger. Matrices with displacement structure: a deterministic approach for linear systems and nullspace bases, 2026. URL: <https://arxiv.org/abs/2603.02425>, arXiv:2603.02425.
- [6] Victor Y. Pan. *Structured Matrices and Polynomials: Unified Superfast Algorithms*. Birkhäuser & Springer, 2001. URL: <https://doi.org/10.1007/978-1-4612-0129-8>.
- [7] Janusz P. Papliński, Aleksandr Cariow, Paweł Strzelec, and Marta Makowska. Digital signal processing (dsp)-oriented reduced-complexity algorithms for calculating matrix–vector products with small-order toeplitz matrices. *Signals*, 5(3):417–437, 2024. URL: <https://www.mdpi.com/2624-6120/5/3/21>, doi:10.3390/signals5030021.
- [8] The FLINT team. *FLINT: Fast Library for Number Theory*, 2025. Version 3.3.1.
- [9] S. S. Zhou and J. B. Wang. Efficient quantum circuits for dense circulant and circulant like operators. *Royal Society Open Science*, 4(5):160906, 05 2017. arXiv:<https://royalsocietypublishing.org/rsos/article-pdf/doi/10.1098/rsos.160906/509313/rsos.160906.pdf>, doi:10.1098/rsos.160906.

Annexe

Implementations

Displacement

```
Project
\  src
|  \  displacement_matrices
|  |  |  displacement_matrices.c
|  |  |  displacement_matrices.h
```

gr_mat_displacement(gr_mat_t D, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function taking as parameter the matrix A and returns into the matrix D , indicating destination, the displacement matrix of A . According to the type of displacement we pass via the parameter **type**, it calculates the implementations (2) or (5).

int gr_mat_G_H(gr_mat_t G, gr_mat_t H, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function that computes the generator matrices G and H^T for an input matrix A . Based on the displacement equations discussed previously, the equation (4), the function first calculates the displacement matrix according to the specified type of displacement. Since the displacement of a structured matrix (like a Toeplitz matrix) inherently results in a matrix with a low rank (α), D can be factored into two smaller matrices such that $D = GH^T$. The function achieves this by performing an LU decomposition on the displacement matrix D . It then extracts the independent columns to form G and the corresponding rows to form H^T . This is where the storage economy for structured matrices come from.

int gr_mat_reconstruct_A_v2(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx); Is the function that reconstructs the original matrix A from its generator matrices G and H .

Compression

```
Project
\  src
|  \  compression
|  |  |  compression.c
|  |  |  compression.h
```

int gr_mat_generator_compress(gr_mat_t G_d, gr_mat_t H_d, gr_ctx_t ctx);
Is our function that implements the algorithms talked in chapter 3.

Utility

```
Project
\  src
|  \  utility
|  |  |  matrix_aux.c
|  |  |  matrix_aux.h
```


int gr_mat_apply_Z(gr_mat_t Res, gr_mat_t M, gr_ctx_t ctx); Function that we implement to apply the downshift matrix Z to a matrix M and store the result in Res .

int gr_mat_apply_Zt(gr_mat_t Res, gr_mat_t M, gr_ctx_t ctx); Function that we implement to apply the transpose of the downshift matrix Z^T to a matrix M and store the result in Res .